COMPLETE TEST SPECIFICATION

PayCaps CRM Complete Application Test Plan

Quality Assurance Coverage for Sandbox Core Payment Engine & Administrative Management Portal

Application Overview & Execution Environment

Complete functional test plan for the PayCaps CRM sandbox application at `https://sandbox.paycaps.com/crm`. Coverage is derived from authenticated exploration using the authorized admin account and manually entered CAPTCHA challenges. Each test executes from a fresh browser context, uses isolated sandbox data, and runs independently and repeatably.

Target Route: https://sandbox.paycaps.com/crm

Environment Mode: Sandbox Pre-Production

Authorized Role: ADMIN (Full Privileges)

Test Isolation: Fresh Browser Context per Test

1. Authentication and Session Security

`tests/seed.spec.ts`

1.1 Authenticate with valid credentials and manually entered CAPTCHA

`tests/auth/login-happy-path.spec.ts`

1. Start fresh browser context and navigate to `https://sandbox.paycaps.com/crm/login`.
   - ✓ Expect: Page shows PayCaps logo, Welcome Back, Email, Password, Captcha, Reload captcha, Forgot Password, and Sign In.
2. Enter authorized email address and password.
   - ✓ Expect: Email accepted, password masked, and no credential value exposed in page text.
3. Enter CAPTCHA manually, then click Sign In.
   - ✓ Expect: Redirected to /crm/dashboard; dashboard header, navigation, account menu, and payment summary visible.

1.2 Validate empty, malformed, incorrect, and stale login submissions

`tests/auth/login-validation.spec.ts`

1. Click Sign In with all fields empty.
   - ✓ Expect: Required validation shown for each mandatory field; request not submitted.
2. Enter malformed email, short/invalid password, and invalid CAPTCHA, then submit.
   - ✓ Expect: Field validation or auth error displayed; CRM remains locked.
3. Refresh CAPTCHA, submit previous challenge value, then re-enter current CAPTCHA and submit.
   - ✓ Expect: Stale value rejected; current CAPTCHA succeeds without browser reload.
4. Click Reload captcha repeatedly.
   - ✓ Expect: New challenge displays each time without corrupting form or sending duplicate requests.

1.3 Validate forgot-password access and recovery behavior

`tests/auth/forgot-password.spec.ts`

1. Select Forgot Password? from login page.
   - ✓ Expect: Opens recovery page with email field and submission control.
2. Submit blank and malformed email values.
   - ✓ Expect: Required and format errors shown; no recovery request accepted.
3. Submit permitted sandbox email address.
   - ✓ Expect: Generic confirmation shown without account-enumeration risk.

1.4 Verify profile, theme, sidebar, logout, and protected-route behavior

`tests/auth/session-lifecycle.spec.ts`

1. Log in, toggle sidebar, toggle theme, and navigate to another module.
   - ✓ Expect: Sidebar collapses/expands smoothly; theme persists without degrading readability.
2. Open User menu, inspect profile route, return to dashboard, then click Logout.
   - ✓ Expect: Menu shows ADMIN role; logout terminates session; direct dashboard access is denied.

2. Global Navigation and Dashboard

`tests/seed.spec.ts`

2.1 Open all primary navigation sections and preserve route context

`tests/navigation/all-primary-navigation.spec.ts`

1. Open primary items: POI, Transactions, Reports, Finance, PBL, Invoice, Analytics, Chargeback, Billing, Merchants, Resellers, Sub-Admins.
   - ✓ Expect: Each control opens route or expands submenu; no broken links, duplicated routes, or inadvertent logouts.
2. Open Payment Reports options (Authorized, Sale, Merchant Visibility, Failed, Refund, Risk, Summary, Settlements).
   - ✓ Expect: Intended route, page heading, breadcrumb, filters, and loading/data/empty state visible.
3. Open Account & Finance (MIS, Downloads) and Analytics (Performance, Weekly, Hourly).
   - ✓ Expect: Finance and analytics pages load with corresponding titles, filters, and charts.
4. Open Payment Billing (Acquirer Routing, VAT/Tax, MDR Settings) and Admin Management pages.
   - ✓ Expect: Configuration and management destinations reachable with correct role permissions and breadcrumbs.

2.2 Verify dashboard metrics, chart, date range, filters, and actions

`tests/dashboard/overview-and-filters.spec.ts`

1. Open Dashboard; inspect Admin payment overview, Today range, Filters, More actions, and Monthly Transaction chart.
   - ✓ Expect: Summary cards (Success, Failed, Invalid, Refund, Success rate) reconcile count/amount totals.
2. Change Date range and apply supported periods; apply dashboard filters individually and combined.
   - ✓ Expect: Cards and charts refresh synchronously; clearing filters restores original baseline.
3. Test More actions cancellation and evaluate a date/filter combination containing zero records.
   - ✓ Expect: Modals cancel without side effects; zero-data ranges display explicit empty states without broken axes.

3. Transactions and Payment Reports

`tests/seed.spec.ts`

3.1 Search, filter, inspect, and paginate transaction records

`tests/transactions/transaction-list-filtering.spec.ts`

1. Open Transactions; search by exact visible TXN ID and ORDER ID.
   - ✓ Expect: Exact matching record displayed; clearing search restores complete listing.
2. Apply date, merchant, status, payment method, transaction type, currency, and amount filters.
   - ✓ Expect: All returned records satisfy combined active filters; invalid queries return clean no-results state.
3. Change rows per page, navigate boundary pagination, and open transaction details.
   - ✓ Expect: Page size and boundary disabled states work; transaction modal matches TXN ID, amount, currency, and date.

3.2 Verify payment report types and consistent report behavior

`tests/reports/report-navigation-and-data.spec.ts`

1. Open each Payment Reports submenu page; check headings, filters, columns, and data states.
   - ✓ Expect: Each report renders dedicated columns and controls consistently.
2. On Sale Report, inspect captured payment columns and masked customer card details.
   - ✓ Expect: TXN ID, order ID, email, method, and amounts match underlying transactions; card PAN numbers remain masked.
3. Apply date/merchant filters and traverse pagination across report views.
   - ✓ Expect: Filtered tables show matching rows; totals reconcile; pagination does not duplicate items.

3.3 Export reports and safely process eligible refunds

`tests/reports/export-and-refund.spec.ts`

1. Apply a known filter to Sale Report and click Export.
   ✓ Expect: Downloadable report generated with filtered columns; clear error notice if export fails.
2. Select Refund on an eligible transaction; review confirmation dialog; test cancel.
   ✓ Expect: Dialog shows TXN ID and amount; canceling leaves transaction unaltered.
3. Confirm refund once, refresh report, and attempt duplicate refund or Refund All without eligible rows.
   ✓ Expect: Processed exactly once; duplicate click blocked idempotently; empty Refund All safely canceled.

3.4 Verify MIS reports and chargeback case management

`tests/reports/mis-and-chargeback.spec.ts`

1. Open Account & Finance > MIS Reports; filter by merchant, acquirer, and date, then export.
   ✓ Expect: Nodal settlement table and export file match gross, commission, payable, and payout values.
2. Open View Chargeback; filter cases by ID, merchant, status, and target date.
   ✓ Expect: Case count updates; matching case details reflect list identifiers; boundary pagination handles smoothly.

4. Pay By Link and Invoice Workflows

`tests/seed.spec.ts`

4.1 Create and validate single- and multi-payment links

`tests/pay-by-link/create-and-validate.spec.ts`

1. Open Pay By Link > Create; test blank submissions and invalid inputs (phone, amount ≤ 0, past expiry, purpose > 100 chars).
   ✓ Expect: Field-level validation blocks submission; invalid entries do not update summary panel.
2. Submit valid Single payment configuration; confirm creation once.
   ✓ Expect: Active link created; displays in table with correct customer, amount, currency, and status.
3. Configure Multi-payment link; test list filters, pagination, and single-record actions.
   ✓ Expect: Multi-payment settings apply cleanly without data leakage; row actions operate strictly on selected record.

4.2 Create an invoice and verify fields, preview, VAT, and totals

`tests/invoices/create-and-calculate.spec.ts`

1. Open Invoice > Create; submit blank and malformed/negative values across required and numeric fields.
   ✓ Expect: Required, format, length, and numeric errors prevent submission.
2. Enter valid merchant, customer details, line items, quantity, amount, expiry, and VAT percentage.
   ✓ Expect: Live preview validates Subtotal + VAT = Total calculation with exact rounding.
3. Test review-step cancellation, then confirm invoice creation once.
   ✓ Expect: Cancel leaves state clean; creation produces single invoice matching all fields in listing.

4.3 Preserve form state and prevent duplicate creation

`tests/forms/unsaved-changes-and-retry.spec.ts`

1. Populate Pay By Link / Invoice forms, navigate Back, and return to creation screen.
   ✓ Expect: Follows documented draft behavior; does not submit silently or duplicate entries.
2. Click Continue repeatedly while request is pending; simulate create API failure after valid data entry.
   ✓ Expect: Submit button debounced/disabled; API errors preserve form data without phantom records.

5. Account, Reseller, and Administration Modules

`tests/seed.spec.ts`

5.1 Search merchant management and open merchant setup

`tests/merchants/merchant-list-and-setup.spec.ts`

1. Open Merchants > Merchant List; filter by merchant ID, business name, email, and status.
   ✓ Expect: Returned records match criteria; search reset restores unfiltered table.
2. Open Setup for active and pending merchants; test invalid vs. valid configuration submissions.
   ✓ Expect: Permitted controls reflect merchant status; invalid configurations blocked; valid data persists upon save.

5.2 Validate merchant registration and merchant sub-user flows

`tests/accounts/user-registration-and-subusers.spec.ts`

1. Open User Registration and Merchant Sub-Users; test empty, duplicate, and invalid role values.
   ✓ Expect: Validation prevents malformed accounts without revealing existing user details.
2. Create, edit, and deactivate permitted sandbox records.
   ✓ Expect: Status updates persist; deactivation requires deliberate confirmation.

5.3 Verify reseller accounts and reseller mapping

`tests/accounts/reseller-management.spec.ts`

1. Open Reseller Setup > Reseller Accounts and Reseller Mapping.
   ✓ Expect: Breadcrumbs, relationship lists, mapping controls, and empty states render accurately.
2. Execute invalid, duplicate, and valid mapping operations in sandbox mode.
   ✓ Expect: Failures surface clear errors; valid mappings associate correctly without bleed between entities.

5.4 Manage sub-admins and review user login history

`tests/accounts/subadmins-and-login-history.spec.ts`

1. Open Sub-Admins > Manage Sub-Admins; test validation on create/edit fields.
   ✓ Expect: Mandatory, email format, and role constraints strictly enforced.
2. Create and deactivate sandbox sub-admin; open User Login History and filter by user/date.
   ✓ Expect: Sub-admin lifecycle reflected; login audit log records accurate usernames, timestamps, and outcomes.

6. Analytics, Payment Configuration, and Access Control

`tests/seed.spec.ts`

6.1 Verify analytics report views and date/filter behavior

`tests/analytics/analytics-reports.spec.ts`

1. Open Analytics reports (Performance, Weekly Performance, Hourly Performance); apply date filters.
   ✓ Expect: Headings, visualizations, and summary cards refresh synchronously; zero-data periods show clean empty states.

6.2 Verify payment billing configuration pages

`tests/configuration/payment-billing-controls.spec.ts`

1. Open Acquirer Routing, VAT / Service Tax, MDR Setting, and MDR Surcharge Setting.
   ✓ Expect: Configuration inputs load current values; invalid range/excess precision submissions are blocked.
2. Save valid settings, test cancellation of unsaved changes, and verify role-based read/write restrictions.
   ✓ Expect: Confirmed saves persist reliably; non-permitted roles cannot access or alter billing parameters.

6.3 Enforce route and action authorization

`tests/access/role-and-route-protection.spec.ts`

1. Access protected URLs in an unauthenticated browser context; test restricted roles against admin actions.
   ✓ Expect: Redirects to login immediately; restricted accounts receive 403 Forbidden or hidden actions.
2. Attempt direct API mutations for refund, creation, and configuration alterations.
   ✓ Expect: Server-side token validation blocks unauthorized calls and enforces mutation idempotency.

7. Quality, Error Handling, Accessibility, and Security

`tests/seed.spec.ts`

7.1 Verify loading, empty, timeout, and service-error handling

`tests/quality/loading-empty-error-states.spec.ts`

1. Simulate slow network latency across list and reporting screens.
   ✓ Expect: Clear loading skeletons/spinners display; partial datasets are never rendered as final.
2. Query zero-result filter combinations across all CRM modules.
   ✓ Expect: Explicit empty-state graphic/message appears with intact headers and functional pagination controls.
3. Inject 4xx, 5xx, and timeout responses using network interception during list, export, create, and save requests.
   ✓ Expect: UI displays actionable error toasts; user-entered form data is preserved; retry actions remain safe.
4. Trigger navigation and page refreshes while background mutation requests are pending.
   ✓ Expect: Client does not execute duplicate submissions; resulting state resolves cleanly without corruption.

7.2 Verify keyboard access, semantics, themes, and responsive layout

`tests/quality/accessibility-responsive.spec.ts`

1. Navigate login, main navigation, filters, dropdowns, tables, and modal dialogs using keyboard only (Tab, Shift+Tab, Enter, Escape).
   ✓ Expect: Focus order is logical; focus outlines are visible; modals trap focus and restore focus on exit.
2. Inspect portal at desktop (1440px), tablet (768px), and mobile (375px) viewport widths.
   ✓ Expect: Layout flexes cleanly; navigation collapses into accessible mobile drawer; tables provide smooth horizontal scrolling.
3. Evaluate Light and Dark themes with automated accessibility color contrast checkers.
   ✓ Expect: Text, badges, input borders, and chart graphics maintain WCAG AA compliant contrast ratios.
4. Audit images, CAPTCHA challenge, icons, and buttons using accessibility screen-reading tools.
   ✓ Expect: Essential images provide informative alt text; icons contain aria-hidden or distinct accessible names.

7.3 Protect sensitive data and safely handle hostile input

`tests/quality/input-security-and-data-protection.spec.ts`

1. Inspect transaction records, reports, refund receipts, invoices, and export CSV/PDF files for PCI data.
   ✓ Expect: Card numbers masked (BIN + ****** + Last 4); raw passwords and CAPTCHA solutions never logged or stored client-side.
2. Inject cross-site scripting (XSS), SQL metacharacters, HTML markup, Unicode symbols, and oversized strings into all inputs.
   ✓ Expect: Values are sanitized and rendered as literal text; backend validations reject malicious payloads without crashes.
3. Execute browser back/forward, duplicate tab actions, and direct URL invocations around refund, create, and save workflows.
   ✓ Expect: Session authorization remains strictly enforced; destructive mutations cannot be accidentally re-executed.
4. Inspect browser network console during failed authentication and unauthorized resource access.
   ✓ Expect: No security tokens, credentials, stack traces, or internal server architecture details are exposed.

Test Automation & Regression Strategy

All test suites referenced above are scripted against Playwright with TypeScript bindings. Automated regression runs must be executed against isolated sandbox merchants to guarantee zero state contamination across concurrent test workers.